

Bitácora de uso de inteligencia artificial

Nombre: Sebastián Cruz

Fecha: 17-05-2026

Asignatura: Programación Front End

IA Utilizada: Google AntiGravity (Gemini Pro 3.1 [high])

- Hito 1

Se le pidió a la IA lo siguiente: “Como un desarrollador de Frontend experto, necesito que crees la estructura base de una página que utilice el método CRUD local en JavaScript. La temática trata sobre registro de usuarios y sus hobbies. Para ello, en primer lugar genera un archivo HTML limpio y semántico que contenga un formulario con tres campos; Nombre completo del usuario, luego su correo y en último lugar el tipo de música que le gusta.”

El resultado fue el siguiente:

Gestión de Usuarios y Hobbies

Registrar Nuevo Usuario

Nombre Completo:

Correo Electrónico:

Tipo de Música que le gusta:

Usuarios Registrados

Nombre Correo Música Favorita Acciones

Luego, se pidió lo siguiente para una mejoría: “Optimiza el código del HTML mediante mejor separación de estructura y contenedores de section con clases semánticas. Añade elementos de texto vacíos con una clase específica para mensajes de error como "error-message" y modifica la sección de "Usuarios registrados" para que, en lugar de texto plano, se estructure usando semánticos de tabla.”

La mejoría puramente de HTML resultó en esto:

Gestión de Usuarios y Hobbies

Registrar Nuevo Usuario

Nombre Completo:

Correo Electrónico:

Tipo de Música que le gusta:

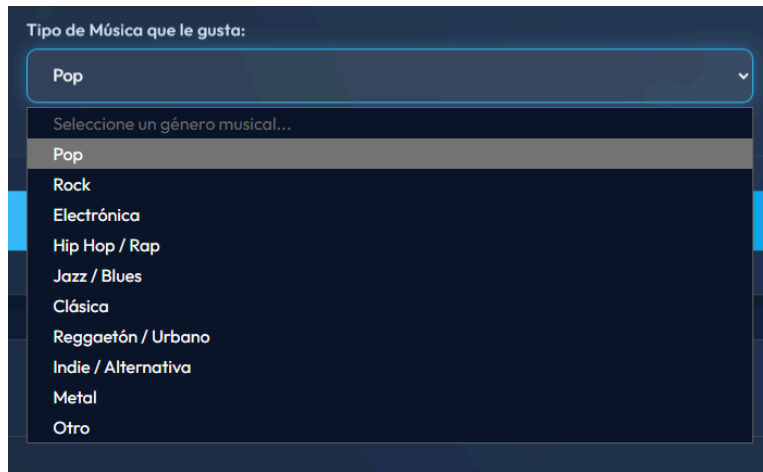
Usuarios Registrados

Nombre Correo Música Favorita Acciones

Como último detalle, debido a un error mínimo con el campo de “Tipo de música que le gusta”, se exigió lo siguiente:

”Necesito una corrección en el archivo HTML con respecto al campo de tipo de música, necesito que, en vez de que sea un input por el usuario, el usuario sea capaz de seleccionar tipos de música generales (o géneros populares) por medio de una tabla en ese campo.“

el resultado fue, como se pidió, una tabla desplegable:

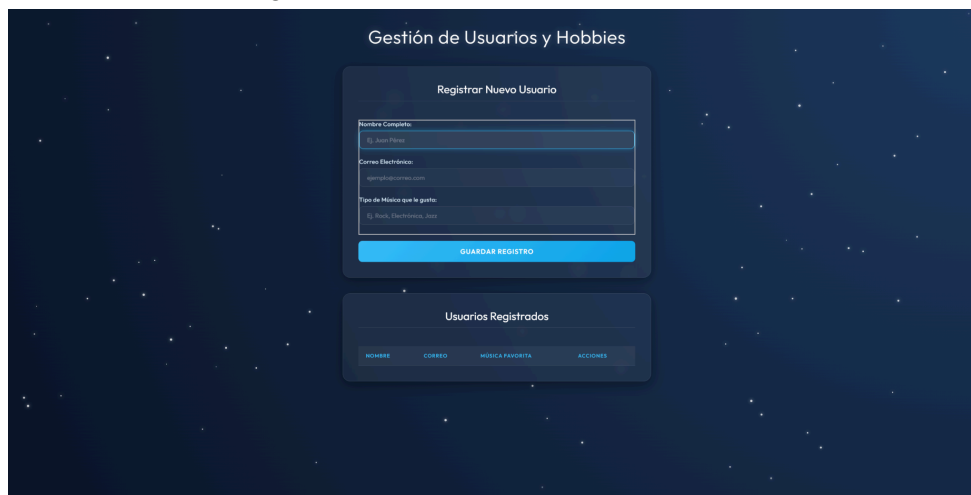


The image shows a dark-themed user interface. At the top, there is a label "Tipo de Música que le gusta:". Below it is a dropdown menu with "Pop" selected. The dropdown is open, showing a list of music genres: "Pop", "Rock", "Electrónica", "Hip Hop / Rap", "Jazz / Blues", "Clásica", "Reggaetón / Urbano", "Indie / Alternativa", "Metal", and "Otro". The background is a dark blue gradient.

- Hito 2

Para el lado de CSS, o en este caso, el aspecto de la página, se pidió lo siguiente: “Ahora, en relación al archivo creado, personaliza/decora la página en lenguaje CSS3, de tal forma que sea suave pero aún llamativo para el usuario, prefiere colores que imiten la temporada de invierno de forma gradual en forma de copos de nieve, y que tengan movimiento dinámico al cursor en pantalla (que se alejen del cursor como un paraguas). “

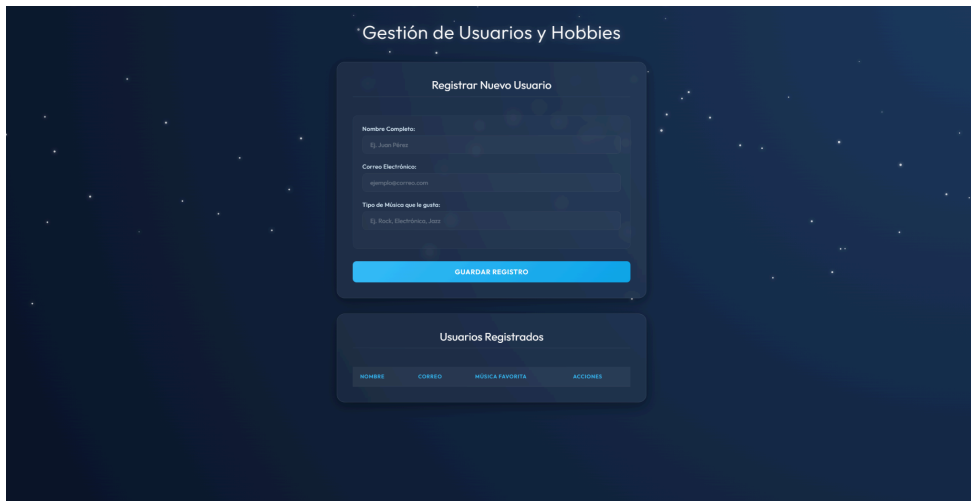
El resultado fué el siguiente:



The image shows a dark-themed user interface with a starry background. The title "Gestión de Usuarios y Hobbies" is at the top. Below it is a form titled "Registrar Nuevo Usuario". The form has three input fields: "Nombre Completo" (with "Juan Pérez" entered), "Correo Electrónico" (with "jgarcia@dominio.com" entered), and "Tipo de Música que le gusta" (with "Rock, Electrónica, Jazz" entered). Below the form is a blue button labeled "GUARDAR REGISTRO". Underneath the form is a section titled "Usuarios Registrados" with a table that has four columns: "NOMBRE", "CORREO", "MUSICA FAVORITA", and "ACCIONES".

Para optimizarlo y que no hayan errores visuales como ese cuadro blanco alrededor de los campos, se pidió lo siguiente: “Quita ese marco blanco alrededor de los campos, e intenta crear un marco que sea menos contrastante con los campos, que combine con el color gris que tiene el cuadro general.”

La mejoría por consecuencia fue:

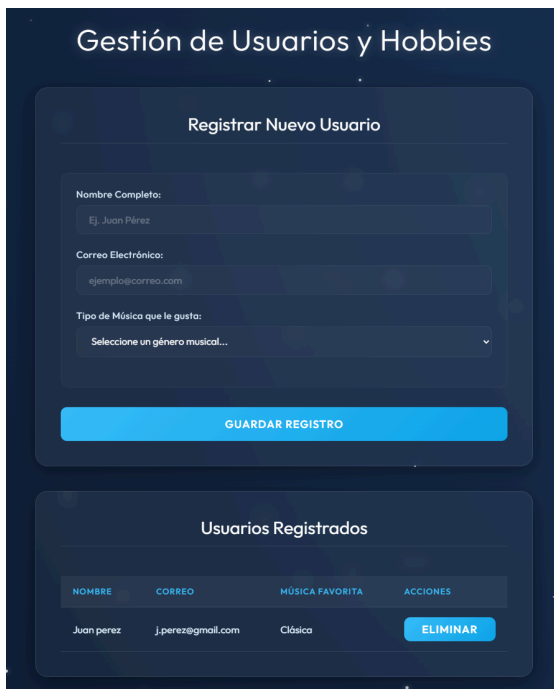


(Bastante más limpio)

- Hito 3

Por último, la funcionalidad de JavaScript. En este caso, se pidió lo siguiente: “Ahora, en base al archivo HTML existente, necesito que completes la lógica mediante una función (llámalo initCRUD). Empieza por crear un arreglo local vacío para almacenar usuarios, y cada usuario debe contener los objetos date, fullname, email y musicType. Además., añade un listener al evento submit del usuario, evita una recarga de la página y captura los valores limpiando los espacios con `.trim()`. En el primer campo de nombre completo, nunca debe estar vacío al hacer submit, en el segundo campo de correo, debe tener formato correspondiente, si existe un error en alguno de estos dos, escribe el mensaje dentro del `<span>` de error, usando estrictamente `.textContent`. Para finalizar, si todos los datos son válidos, guarda el objeto en el arreglo, limpiando el formulario con un `.reset()`, y después, llama a una nueva función llamada `renderUsers()` para actualizar la tabla. La función `renderUsers()` debe limpiar el `<tbody>`, luego, usa `document.createElement()` y `.textContent` para insertar filas de manera segura. Al final de todo esto, agrega un botón de "Eliminar" en la columna de "Acciones" para que el usuario tenga la posibilidad de remover a un usuario del arreglo usando `.filter()`, y que vuelva a renderizar la tabla luego de eso.”

El resultado fue el siguiente:



Para darle una optimización robusta y seguridad, se pidió lo siguiente:” Ahora, de nuevo en JavaScript, optimiza y refactoriza el código que se acaba de construir en base al método CRUD, bajo los siguientes estándares:

1.- Implementa expresiones regulares estrictas para validar que fullname contenga solo letras y espacios (y un mínimo de 3 caracteres), y otra para el campo email. Si la validación falla, añade una clase en el archivo CSS de error al input (clásico ejemplo de error de color rojo) además de mostrar el texto en el span.

2.- Analiza el renderizado dinámico para asegurar que, bajo ningún caso, se use innerHTML con datos del usuario, todo debe ser mapeado con nodos nativos y .textContent.

3.- Integra un localStorage para que los usuarios registrados se mantengan guardados si la página se recarga (por accidente usualmente). Al iniciar el DOMContentLoaded, lee el almacenamiento local y pinta los usuarios existentes.

4.- Por último, añade un botón de "Editar" al lado de "Eliminar" en la sección de "Acciones". Al presionarlo, debe cargar lo datos del usuario seleccionado de vuelta en los inputs del formulario para permitir su modificación, cambiando temporalmente el comportamiento del botón principal a "Actualizar". “

El resultado fue este:

The screenshot shows a web application interface with a dark blue background. At the top, the title "Gestión de Usuarios y Hobbies" is displayed in white. Below the title, there are two main sections:

Registrar Nuevo Usuario

This section contains a form with three input fields and a dropdown menu:

- Nombre Completo: Sebastián Cruz
- Correo Electrónico: hailgo20@gmail.com
- Tipo de Música que le gusta: Hip Hop / Rap (selected from a dropdown)

Below the form is a large blue button labeled "ACTUALIZAR".

Usuarios Registrados

This section displays a table of registered users:

NOMBRE	CORREO	MÚSICA FAVORITA	ACCIONES
Sebastián Cruz	hailgo20@gmail.com	Hip Hop / Rap	<button>EDITAR</button> <button>ELIMINAR</button>

Gestión de Usuarios y Hobbies

Registrar Nuevo Usuario

Nombre Completo:

Ej. Juan Pérez

Correo Electrónico:

ejemplo@correo.com

Tipo de Música que le gusta:

Ej. Rock, Electrónica, Jazz

GUARDAR REGISTRO

Usuarios Registrados

NOMBRE	CORREO	MÚSICA FAVORITA	ACCIONES
Sebastián Cruz	hailgo20@gmail.com	Hip Hop / Rap	EDITAR ELIMINAR

(Aquí salí del navegador y volví a entrar, se puede notar que almacenó el usuario guardado sin problemas)